Data Engineer Aassignment 1

Data Engineering Assignment: Employee Data Pipeline

Overview

This assignment tests your ability to build an end-to-end data pipeline using Apache Spark and PostgreSQL, with a focus on data cleaning, transformation, and loading operations. All components should run in Docker containers.

Objectives

- Generate and work with sample CSV data
- Perform data cleaning and transformations using Apache Spark
- Load processed data into PostgreSQL database
- Demonstrate proficiency with Docker, Spark, and SQL

Assignment Tasks

Task 1: Environment Setup

Set up a Docker environment with the following services:
- Apache Spark
- PostgreSQL database

Requirements:
- Create a `docker-compose.yml` file to orchestrate all services
- Ensure Spark can connect to PostgreSQL
- Include necessary JDBC drivers for PostgreSQL connectivity

Task 2: Sample Data Generation

Create a CSV file named `employees_raw.csv` with the following structure and at least 1000 records:

```
employee_id,first_name,last_name,email,hire_date,job_title,department,salary,
manager_id,address,city,state,zip_code,birth_date,status
```

Sample Data Characteristics:
- Include various data quality issues (missing values, duplicates, inconsistent formats)
- Mix of valid and invalid email formats
- Some future hire dates (data errors)
- Salary values with currency symbols and commas
- Mixed case in categorical fields
- Some null/empty values in non-critical fields

Data Generation Requirements:
You can use Python, Faker library, or any tool of your choice to generate this sample data.

Task 3: Data Cleaning and Transformation

Write a Spark job (Scala preferred) that performs the following operations:

3.1 Data Quality Checks

- Identify and handle missing values
- Remove duplicate records based on employee_id
- Validate email formats
- Check for logical inconsistencies (e.g., hire_date > current_date)

3.2 Data Transformations

- **Name Standardization**: Convert first_name and last_name to proper case
- **Email Cleanup**: Ensure all emails are lowercase and valid format
- **Salary Cleaning**: Remove currency symbols, commas, and convert to numeric
- **Date Validation**: Ensure hire_date is not in the future
- **Age Calculation**: Calculate employee age from birth_date
- **Tenure Calculation**: Calculate years of service from hire_date

- **Salary Bands**: Create salary bands (Junior: <50k, Mid: 50k-80k, Senior: >80k)

3.3 Data Enrichment

- Add a `full_name` column (first_name + last_name)
- Create an `email_domain` column extracted from email

Task 4: Database Design and Loading

4.1 Database Schema

Design and create PostgreSQL tables:

Main Table: `employees_clean`

```
CREATE TABLE employees_clean (
    employee_id INTEGER PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    email_domain VARCHAR(50),
    hire_date DATE NOT NULL,
    job_title VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL(10,2),
    salary_band VARCHAR(20),
    manager_id INTEGER,
    address TEXT,
    city VARCHAR(50),
    state VARCHAR(2),
    zip_code VARCHAR(10),
    birth_date DATE,
    age INTEGER,
    tenure_years DECIMAL(3,1),
    status VARCHAR(20) DEFAULT 'Active',
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

4. 2 Data Loading

- Load the cleaned employee data into the `employees_clean` table
- Implement proper error handling and logging

Task 5: Documentation and Code Organization

Provide the following deliverables:

1. **README.md** with setup instructions
2. **docker-compose.yml** for environment setup
3. **Spark job code** (well-commented Python/Scala)
4. **SQL scripts** for database setup
5. **Data generation script**
6. **Sample input and output data**

Evaluation Criteria

Technical Implementation (40%)

- Correct Spark transformations and optimizations
- Proper error handling and logging
- Code organization and best practices
- Database design and constraints

Data Quality (25%)

- Comprehensive data cleaning logic
- Appropriate handling of edge cases
- Data validation and consistency checks

Docker & Infrastructure (20%)

- Proper containerization
- Service orchestration
- Network configuration
- Resource management

Documentation (15%)

- Clear setup instructions
- Code comments and documentation
- Data pipeline explanation
- Troubleshooting guide

Submission Guidelines

1. Create a Git repository with all code and documentation
2. Include a demo video (5 minutes or so) showing the pipeline execution
3. Provide sample input data and expected output

Time Expectation

This assignment should take approximately 2 days to complete, including setup, development, testing, and documentation.

Sample Employee Data Structure

Here's the expected structure for your raw input CSV:

```
employee_id,first_name,last_name,email,hire_date,job_title,department,salary,manager_id,address,city,state,zip_code,birth_date,status
1001,john,DOE,John.Doe@company.com,2020-01-15,Software Engineer,IT,"$75,000",2001,123 Main St,New York,NY,10001,1990-05-15,Active
1002,jane,smith,jane.smith@COMPANY.COM,2019-03-20,Data Analyst,Analytics,65000,2002,456 Oak Ave,Los Angeles,CA,90210,1988-08-22,Active
```

```
1003,Bob,johnson,bob@company,2028-01-01,Manager,IT,"$95,000",,789 Pine Rd,Chicago,IL,60601,1985-12-10,Active
```

Note: This sample shows the types of data quality issues you should include in your generated dataset.

Questions?

If you have any questions about the assignment requirements, please don't hesitate to ask for clarification.

Good luck!